

FASTROUTE KAYSERI

Urban Logistics & Distribution System

COMP206

Semester Project Report

Eren Bülbül

Tuğçe Taşcı

Zeynep Aybuke Ilık

Ozan Akdeniz

Hamza Mete Erbölükbaş

1. Company Name

The company name of this project is **FastRoute Kayseri**.

FastRoute Kayseri is designed as a logistics and distribution company that manages package flow inside a warehouse and optimizes delivery routes across different neighborhoods in Kayseri. The company name is also included in the console output and the final report to satisfy the project identity requirement.

2. Mission Statement

FastRoute Kayseri aims to improve package delivery and warehouse management operations in Kayseri by using efficient data structures and graph algorithms. The system organizes daily package records, manages incoming package flow, prepares packages for delivery, supports truck loading, stores address information, and optimizes city routes.

The main mission of FastRoute Kayseri is to provide a fast, reliable, and organized logistics service. To achieve this goal, the system selects the most suitable data structure for each operational task. For example, linked lists are used for package records and buffer management, a queue is used for standard delivery order, a stack is used for truck loading, an AVL tree is used for address searching, and a weighted graph is used for city routing.

Complexity Analysis Table

Component	Operation	Time Complexity	Explanation
Singly Linked List	addRecord(Package pkg)	$O(n)$	Traverses the list to append a package at the end.
Singly Linked List	displayLog()	$O(n)$	Visits every package record once.
Singly Linked List	isEmpty()	$O(1)$	Checks only the head pointer.
Doubly Linked List	insertAtTail(Package pkg)	$O(1)$	Uses the tail pointer to insert directly at the end.
Doubly Linked List	removeFromHead()	$O(1)$	Updates the head pointer directly.
Doubly Linked List	displayBuffer()	$O(n)$	Traverses all buffer nodes once.
Queue	enqueue(Package pkg)	$O(1)$	Uses the rear pointer for direct insertion.
Queue	dequeue()	$O(1)$	Uses the front pointer for direct removal.
Queue	displayQueue()	$O(n)$	Visits every queue node once.
Stack	push(Package pkg)	$O(1)$	Inserts directly at the top.
Stack	pop()	$O(1)$	Removes directly from the top.
Stack	displayStack()	$O(n)$	Visits every stack node once.
AVL Tree	insert(String neighborhood, String customerID)	$O(\log n)$	Tree height remains balanced after rotations.
AVL Tree	search(String neighborhood)	$O(\log n)$	Balanced tree reduces search height.
AVL Tree	displayInOrder()	$O(n)$	Visits every tree node once.
Graph	addEdge(String source, String destination, int weight)	$O(V)$	May search vertex names before adding the edge.
Graph	displayGraph()	$O(V^2)$	Scans the adjacency matrix upper triangle (all vertex pairs).

Component	Operation	Time Complexity	Explanation
Graph	calculateShortestPath(start, end) - Dijkstra	$O(V^2)$	Uses simple loops to select the next closest vertex.
Graph	calculateMST() - Prim	$O(V^2)$	Repeats V times: selects the unvisited vertex with minimum key in $O(V)$, then scans all neighbors in $O(V)$.
File Loading	Read mapData.txt	$O(m)$	Reads each map line once.
File Loading	Read packageData.txt	$O(p)$	Reads each package record once, where p is the number of package records.

n: Number of elements in a list, queue, stack, or tree

V: Number of vertices (neighborhoods) in the graph

m: number of road/map records in mapData.txt

p: Number of valid package records read from packageData.txt